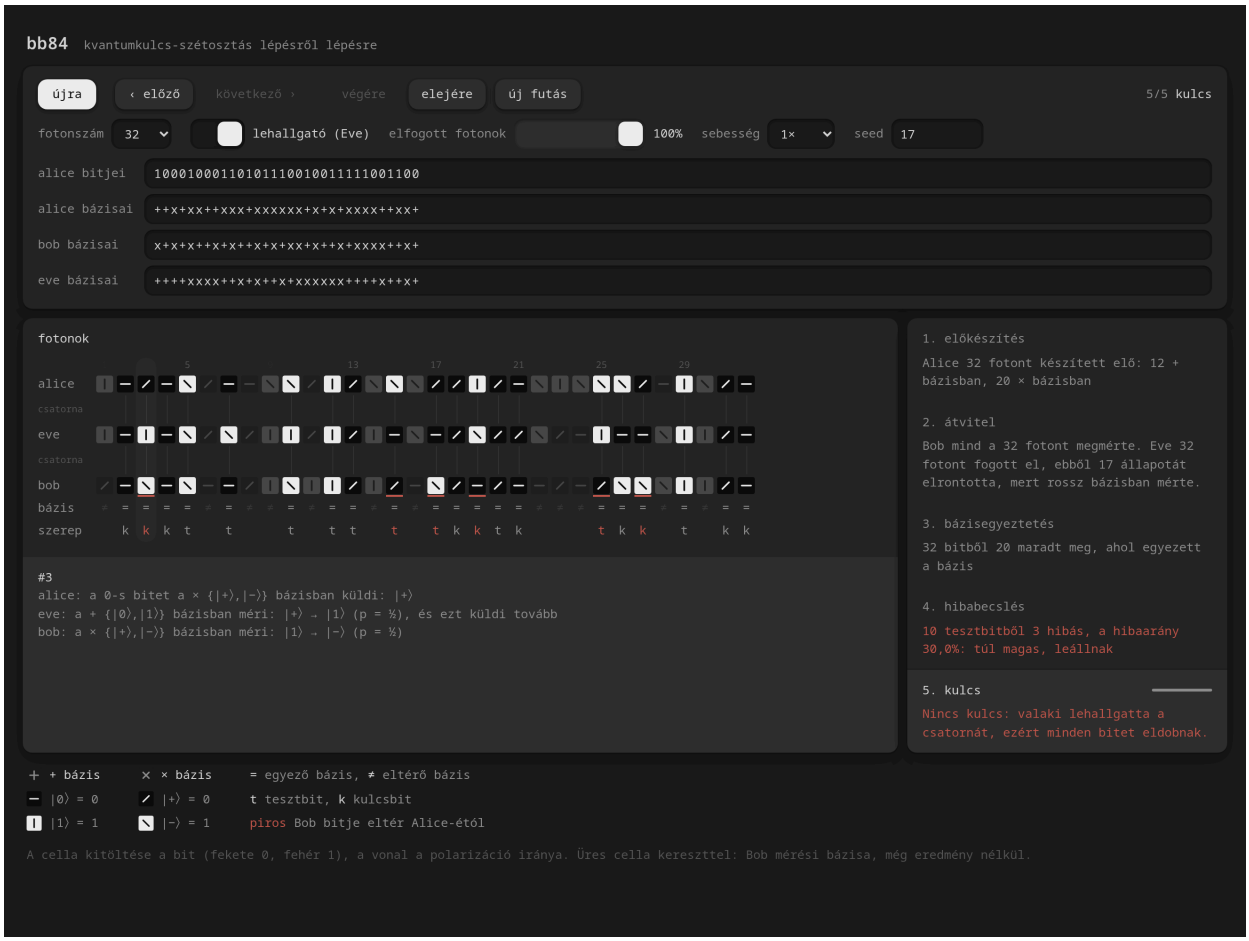
BB84 kvantumkulcs-szétosztás szimulátor

Dokumentáció

Kvantuminformatikai alkalmazások, házi feladat (4. feladat: BB84-szimulátor), 2026 ősz

Készítette (Neptun-kód): H17GP0, P33DEX

A program a BB84 kvantumkulcs-szétosztó (QKD) protokollt szimulálja böngészőben. A protokoll öt szakaszra bontva, animálva követhető, a bitek és bázisok kézzel is megadhatók, és bekapcsolható egy lehallgató (Eve), akinek a hatása a hibaarányon közvetlenül látszik.



1. ábra: A felület egy lefutott protokoll után, teljes lehallgatással. A hibabecslés 30%-os hibaarányt mér, ezért nem jön létre kulcs.

1. A BB84 protokoll röviden

1.1. Bázisok és kódolás

Alice egyedi fotonok lineáris polarizációjába kódol biteket, két bázis egyikében:

1. táblázat: A BB84 négy állapota

bázis	0 bit	1 bit
+ (rectilineáris)	$0° \longrightarrow \lvert 0\rangle$	$90° \longrightarrow \lvert 1\rangle$
× (diagonális)	$45° \longrightarrow \lvert +\rangle$	$135° \longrightarrow \lvert -\rangle$

A szimulátorban egy foton állapota a polarizáció szöge (`photon.ts`): `bázis-eltolás + 90° · bit`, ahol az eltolás + bázisnál 0°, × bázisnál 45°.

1.2. Mérés

Egy θ szögű fotont a φ tengelyű bázisban mérve a mérési posztulátum (Born-szabály) szerint a 0 kimenet valószínűsége

$$P(0) = \cos^2(\theta - \varphi), \quad P(1) = 1 - P(0).$$

Azonos bázisban mérve tehát a kimenet biztos, és visszakapjuk a kódolt bitet. A másik (konjugált) bázisban mérve mindkét kimenet ½ valószínűségű. A mérés után a foton a mért állapotba kerül, az eredeti állapot elvész. A program `measure` függvénye ezt valósítja meg: kisorsolja a kimenetet, és visszaadja a mérés utáni állapotot és a kimenet valószínűségét.

1.3. A protokoll lépései

1. **Előkészítés:** Alice minden fotonhoz véletlen bitet és véletlen bázist választ, és elkészíti a megfelelő állapotot.
2. **Átvitel:** a fotonok a kvantumcsatornán Bobhoz jutnak, aki mindegyiket egy véletlenül választott bázisban méri.
3. **Bázisegyeztetés (sifting):** nyilvánosan egyeztetik a bázisokat (a biteket nem). Ahol a bázis eltér, a bitet eldobják; átlagosan a fotonok fele marad meg.
4. **Hibabecslés:** a megmaradt bitek egy véletlen részhalmazát (a programban a felét) nyilvánosan összehasonlítják. Az eltérések aránya a QBER (quantum bit error rate). Ha ez meghaladja a programban beállított 11%-os küszöböt, a protokollt megszakítják.
5. **Kulcs:** a nem nyilvánosságra hozott, egyező bázisú bitek adják a közös kulcsot.

1.4. Miért 25% körüli a hibaarány lehallgatáskor?

Az elfog-újraküld (intercept-resend) támadásnál Eve minden fotont egy véletlen bázisban megmér, és a mért állapotot küldi tovább. A bázisegyeztetés után csak azok a bitek maradnak, ahol Alice és Bob bázisa azonos. Ezeknél:

- ½ valószínűséggel Eve eltalálja Alice bázisát, a mérése helyes, a továbbküldött foton azonos Alice-éval, hiba nem keletkezik;
- ½ valószínűséggel rossz bázisban mér, a foton a másik bázis egyik állapotába kerül, így Bob mérése ½ valószínűséggel hibás.

$$P_{\text{hiba}} = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4} = 25\%.$$

Ha Eve csak a fotonok r hányadát fogja el, a várható QBER $r/4$.

2. Használat

2.1. Indítás

A beadott `dist/index.html` egyetlen, önálló fájl: dupla kattintással megnyílik az alapértelmezett böngészőben (Windows alatt is), telepítés, szerver és internetkapcsolat nélkül, mert minden JavaScript és CSS a fájlba van ágyazva. Forrásból így építhető (Node.js és pnpm szükséges):

```
pnpm install
pnpm build    # eredmény: dist/index.html
pnpm dev      # fejlesztői szerver
```

2.2. Kezelőszervek



2. ábra: A vezérlőpanel. Az „alice bázisai” mezőben a kiemelt karakter a blokk-kurzor.

- **lejátszás / szünet / újra:** a szakaszok automatikus lejátszása, közöttük rövid szünettel; a végén az „újra” elölről indítja.
- **‹ előző / következő ›:** egy szakasz visszalépése, illetve a következő animálása. Futó animációnál a „következő ›” azonnal befejezi az aktuális szakaszt.
- **végére / elejére:** az összes szakasz eredményének megjelenítése, illetve vissza az elejére ugyanazzal a futással.
- **új futás:** új véletlen seed, azaz új bitek és bázisok. Jobb oldalt az aktuális szakasz sorszáma és neve látható.
- **fotonszám:** a küldött fotonok száma (16, 32, 64 vagy 128).
- **lehallgató (Eve)** és **elfogott fotonok:** a lehallgató be- és kikapcsolása, illetve hogy a fotonok hány százalékát fogja el (0 és 100% között, 5%-os lépésekben; a futás a csúszka elengedésekor számolódik újra).
- **sebesség:** az animáció sebessége (0,5× és 8× között).
- **seed:** a véletlenszám-generátor kezdőértéke. Ugyanaz a seed mindig ugyanazt a futást adja, ezért ha egy futás érdekes, érdemes felírni a seedjét.

Bármely beállítás módosításakor a futás újraszámolódik, de a már lejátszott szakaszok láthatók maradnak. Eve bekapcsolásakor például ugyanazon a futáson rögtön látszik, mi változott. A jobb oldali szakaszlistában egy szakaszra kattintva az előtte lévők eredménye megjelenik, és a kiválasztott szakasz újra lejátszódik.

2.3. Kézi bit- és bázisbevitel

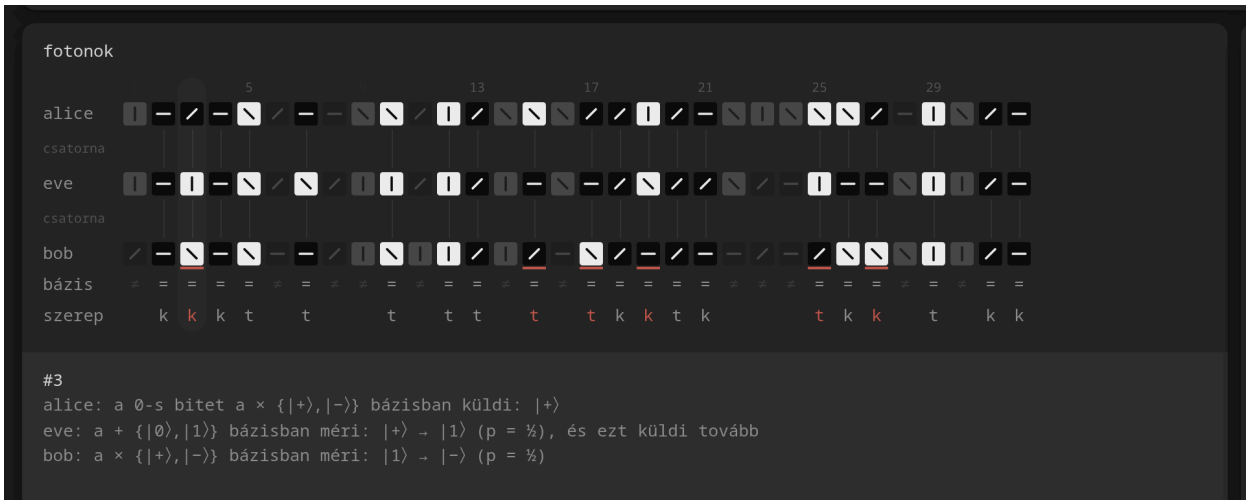
A vezérlőpanel alján négy mező van: **alice bitjei**, **alice bázisai**, **bob bázisai** és **eve bázisai**. Mindegyik az aktuális futás tényleges értékeit mutatja, fotononként egy karakterrel. Bitnek `0` vagy `1`, bázisnak `+` vagy `x` adható meg (az `X` és a `×` is elfogadott). Eve mezőjében a `-` azokat a fotonokat jelöli, amelyeket Eve átenged; ezeken nincs mit megadni. Kikapcsolt lehallgatásnál ez a mező nem szerkeszthető.

A mezők úgy működnek, mint az egyszer használatos kódok (OTP) beviteli mezői: a hossz mindig a fotonszám, törölni vagy beszúrni nem lehet, csak felülírni. Gépeléskor a kiemelt karakter cserélődik le, és a kurzor továbblép (Eve `-` helyeit átugorja); érvénytelen karakterre a mező kerete röviden pirosra vált. A Backspace és a nyilak csak a kurzort mozgatják. Hosszabb sorozat beillesztéssel is megadható, ilyenkor a szóközök és az érvénytelen karakterek kimaradnak.

Minden módosítás azonnal érvénybe lép, a nem módosított mezők a véletlen értékeket használják tovább. Az **új futás** és a fotonszám módosítása törli a kézi értékeket.

2.4. Fotonok követése

A „fotonok” panelen minden oszlop egy foton. Az átvitel után egy oszlop fölé állva a panel alján megjelenik, mi történt az adott fotonnal, a kimenetek valószínűségével együtt. A 3. ábrán Alice $|+\rangle$ állapotot küld, Eve rossz bázisban mér, és $|1\rangle$-et küld tovább, ezért Bob ½ valószínűséggel hibás bitet kap, és ebben a futásban pont ez történt (piros `k`). Egy cella fölé állva rövid felirat mutatja az állapotot (pl. „$|+\rangle$, bit: 0”).



3. ábra: Egy foton útja: Eve rossz bázisban mér, és ezzel hibát okoz Bob bitjében.

2.5. Billentyűparancsok

2. táblázat: Billentyűparancsok

billentyű	művelet
Szóköz	lejátszás / szünet
→ / ←	következő / előző szakasz
E	végére
R	elejére
N	új futás

Ha egy beviteli mezőn, listán vagy csúszkán van a fókusz, a parancsok nem élnek, hogy gépelés közben ne lépjen tovább a lejátszás.

3. Az egyes szakaszok megjelenítése

A „fotonok” panel sorai: **alice**, **csatorna**, **eve** (csak lehallgatásnál, utána újabb csatorna), **bob**, **bázis** (egyezik-e) és **szerep** (teszt- vagy kulcsbit). A cellák jelölését a felület alján lévő jelmagyarázat foglalja össze: a kitöltés a bit, a vonal a polarizáció iránya. Minden szakasz befejezésekor a jobb oldali panelen megjelenik az eredménye.

1. **Előkészítés:** megjelennek Alice állapotai; az eredmény a két bázisban készült fotonok száma.
2. **Átvitel:** a fotonok lefelé haladnak a csatornán. Bob sorában közben csak a mérési bázisa látszik (+ vagy × jel, kimenet nélkül), a végén a mért bit. Lehallgatásnál a fotonok megállnak Eve sorában: ahol elfogta őket, az ő mért állapota halad tovább, ahol átengedte, ott pont (·) látható. Az eredmény megadja, hány fotont fogott el Eve, és ebből hánynak az állapotát rontotta el.
3. **Bázisegyeztetés:** minden oszlop alá `=` vagy `≠` kerül, az eltérő bázisú oszlopok elhalványulnak; az eredmény a megmaradt bitek száma.
4. **Hibabecslés:** a tesztbitek alá `t` kerül. Ahol Bob bitje eltér Alice-étól, a jel piros, és Bob cellája alá piros vonal kerül. Az eredmény a hibás tesztbitek száma, a hibaarány és a döntés (folytatás, vagy 11% fölött leállás).
5. **Kulcs:** a kulcsbitek alá `k` kerül, és Alice és Bob kulcsa egymás alatt megjelenik; az eltérő bitek piros keretet kapnak. Az eredmény a kulcs hossza, az, hogy a két kulcs megegyezik-e, és lehallgatásnál az, hogy Eve hány kulcsbitet ismer. Leállásnál nem jön létre kulcs (1. ábra).



4. ábra: Az átvitel animáció közben: a fotonok Eve sorában vannak, Bob sorában még csak a mérési bázisok látszanak.

A program a hibajavítást (reconciliation) és a bizalmasságnövelést (privacy amplification) nem szimulálja; a kulcs szakasz a hibabecslés után megmaradt nyers kulcsot mutatja.

4. A támadó

A **lehallgató (Eve)** kapcsolóval bekapcsolható támadás az elfog-újraküld támadás (`bb84.ts`, `transmit` függvény):

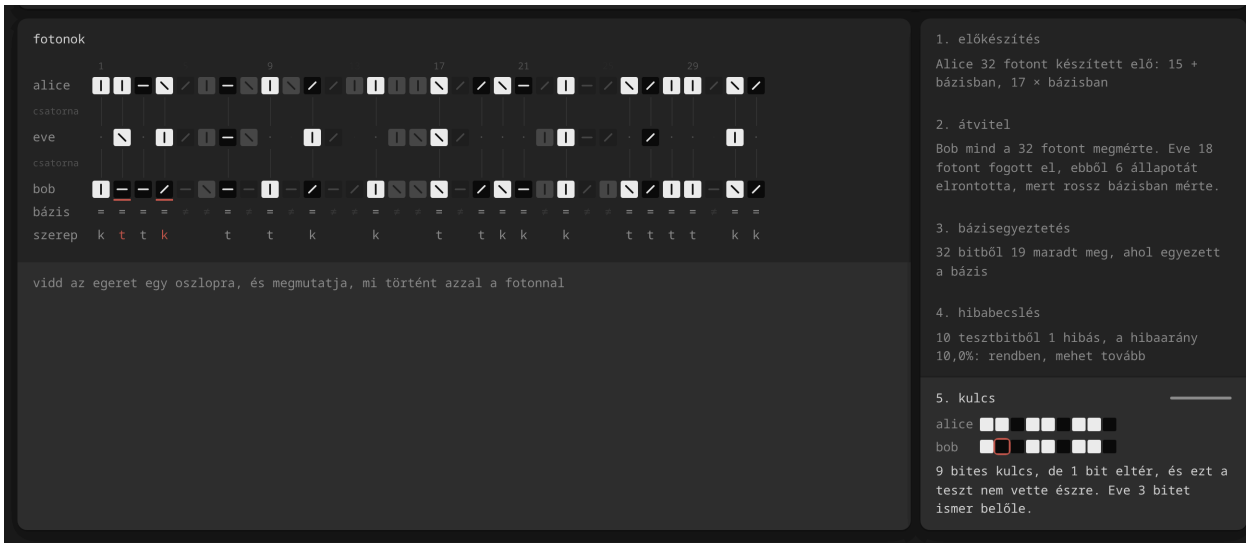
1. Eve minden fotonnál húz egy véletlen számot; ha ez kisebb az **elfogott fotonok** arányánál, elfogja a fotont, különben átengedi.
2. Az elfogott fotont egy véletlen (vagy kézzel megadott) bázisban megméri.
3. A mért bitet a saját bázisában újra kódolja, és ezt az új fotont küldi tovább Bobnak.

A várható QBER az 1.4. pont szerint az elfogott arány negyede. A program saját motorjával, 128 fotonnal és 200 különböző seeddel mért átlagos hibaarány és a megszakított futások aránya:

3. táblázat: A lehallgatás hatása (128 foton, 200 futás átlaga)

elfogott fotonok	átlagos QBER	megszakított futások
0%	0,0%	0%
20%	5,0%	11%
40%	10,7%	46,5%
60%	14,9%	70%
80%	20,3%	88%
100%	25,0%	96,5%

Teljes lehallgatásnál tehát szinte mindig leáll a protokoll (1. ábra). Kis fotonszámnál viszont a tesztminta kicsi (128 fotonnál kb. 32 tesztbit), ezért a becslés szór, és egy lehallgatott futás véletlenül a küszöb alatt maradhat. Az 5. ábrán Eve a fotonok felét fogja el. A 10 tesztbitből csak 1 hibás (10%), ez épp a küszöb alatt van, és a protokoll folytatódik. A kulcsban viszont marad egy eltérés, amit a teszt nem vett észre, Eve pedig a 9 kulcsbitből 3-at ismer.



5. ábra: 50%-os lehallgatás: a hibabecslés átengedi a futást, de a kulcsban észrevétlen eltérés marad.

5. Architektúra és technológia

5.1. Technológiák

- **Svelte 5:** felhasználói felület, „runes” reaktivitással (`$state`, `$derived`, `$props`).
- **TypeScript:** a teljes kód típusos.
- **Vite:** fejlesztői szerver és buildeszköz.
- **vite-plugin-singlefile:** a build során minden JavaScriptet és CSS-t az `index.html`-be ágyaz, ettől lesz a kész program egyetlen fájl.

5.2. Mappaszerkezet

```
index.html                belépési pont
vite.config.ts            Vite + Svelte + singlefile beállítás
src/
  main.ts                 az App csatolása az oldalhoz
  App.svelte              elrendezés, billentyűparancsok
  app.css                 közös stílusok, színek
  components/
    Controls.svelte       gombok, beállítások, kézi bevitel
    FixedInput.svelte     rögzített hosszú, felülíró beviteli mező
    Ledger.svelte         „fotonok” panel, animáció
    Stages.svelte         szakaszlista és eredmények
    Legend.svelte         jelmagyarázat
    Glyph.svelte          egy állapot/bit rajza
    Rich.svelte           ket-jelölés (|0)) megjelenítése
    Tooltip.svelte        közös felugró felirat
  lib/quantum/            a fizika és a protokoll, felülettől független
    photon.ts             kódolás, Born-szabály, mérés
    bb84.ts               a teljes protokoll (runBb84)
    rng.ts                mulberry32 PRNG és a résztvevők folyamai
    bb84.test.ts          tesztek
  lib/ui/
    player.svelte.ts      lejátszás vezérlése (szakaszok, animáció)
    format.ts             feliratok, leírások szövege
```

5.3. Működés

A számítás és a megjelenítés külön van. A `runBb84(config)` tiszta függvény: ugyanarra a konfigurációra mindig ugyanazt adja, és egyszerre kiszámolja a teljes futást: minden foton rekordját (Alice bitje és bázisa, Eve művelete, a Bobhoz érkező állapot, Bob bázisa és mért bitje, a bit szerepe), a tesztmintát, a QBER-t és a kulcsokat.

A `Player` osztály (`player.svelte.ts`) csak azt dönti el, mennyi látszik ebből. A pozíció egyetlen szám a $[0, 5]$ intervallumban: az egész rész a befejezett szakaszok száma, a tört rész az aktuális animáció előrehaladása. A léptetés, a visszalépés és a lejátszás mind ezt az egy számot állítja.

5.4. Véletlenszámok

Véletlenszám-generátornak a **mulberry32**-t választottuk (Tommy Ettinger algoritmusa). Néhány soros, ezért külső csomag helyett egyszerűen a kódba került (`rng.ts`), és seedelhető, tehát minden futás pontosan visszajátszható. Valódi QKD-hez persze nem lenne elég: ott a bit- és bázisválasztásokhoz valódi (vagy legalább kriptográfiailag biztonságos) véletlenforrás kell.

Egy seedből öt külön folyam származik (`streams`): `alice`, `bob`, `eve`, `nature` (mérési kimenetek) és `publicChannel` (a tesztminta kiválasztása). Azt akartuk, hogy Eve be- és kikapcsolása ne változtassa meg Alice és Bob választásait, ezért minden résztvevő minden fotonnál húz a saját folyamából, Eve akkor is, ha ki van kapcsolva. Kézi bevitelnél ugyanez a helyzet: a program a véletlen értéket ekkor is kihúzza, és csak utána cseréli le a megadottra.

6. Tesztek és eredmények

A tesztek a `src/lib/quantum/bb84.test.ts` fájlban vannak, a Node.js beépített tesztkeretével (`node:test`, `node:assert`), külön függőség nélkül. Futtatás: `pnpm test`. A TypeScript-fájlokat a Node.js közvetlenül futtatja, ehhez Node.js 22.18 vagy újabb kell (mi 22.22-t használtunk).

4. táblázat: Automatikus tesztek

teszt	ellenőrzés
lehallgatás nélkül	2000 qubit: QBER = 0, nincs leállás, Alice és Bob kulcsa azonos
teljes lehallgatás	4000 qubit, 100%: QBER 20% és 30% között, a protokoll leáll
bázisegyeztetés	2000 qubit: a megtartott bitek aránya 40% és 60% között
determinizmus	ugyanaz a konfiguráció azonos eredményt ad
kézi bevitel	a megadott bitek és bázisok (a × karakterrel együtt) érvényesülnek
érvénytelen karakter	érvénytelen karakter helyén a véletlen érték marad
azonos bázisú mérés	mind a 4 állapotnál a kódolt bitet adja, 1 valószínűséggel
konjugált bázisú mérés	4000 mérésből az 1-esek aránya 45% és 55% között, a valószínűség ½

Mind a 8 teszt sikeres. A rögzített seeddel mért értékek: lehallgatás nélkül a QBER 0, a megtartott arány 47,3%; teljes lehallgatásnál 970 tesztbitből 251 hibás, azaz a QBER 25,9%, ez közel van az elméleti 25%-hoz. A típusellenőrzés (`pnpm check`: svelte-check és tsc) hiba és figyelmeztetés nélkül fut, a build (`pnpm build`) egyetlen, kb. 77 kB-os `dist/index.html` fájlt állít elő.

7. Felhasznált könyvtárak és források

7.1. Könyvtárak

A csomagok csak a fejlesztéshez kellenek, a kész fájl futáskor semmit nem tölt be.

5. táblázat: Felhasznált csomagok

csomag	verzió	szerep
`svelte`	5.57.3	felhasználói felület
`vite`	8.3.7	build és fejlesztői szerver
`@sveltejs/vite-plugin-svelte`	7.3.1	Svelte-támogatás a Vite-hoz
`vite-plugin-singlefile`	2.3.4	egyfájlos kimenet
`typescript`	6.0.3	típusos nyelv
`svelte-check`	4.7.8	típusellenőrzés
`@tsconfig/svelte`	5.0.8	TypeScript-beállítások
`@types/node`	24.19.2	Node-típusok a tesztekhez

7.2. Források

1. Kvantuminformatikai alkalmazások: jegyzet a tárgy hallgatói számára, 2026 ősz. A tárgy oktatói. (2. Posztulátumok, 4. Mérés, 7. QKD fejezet.)
2. Svelte dokumentáció: https://svelte.dev/docs
3. Vite dokumentáció: https://vite.dev
4. vite-plugin-singlefile: https://github.com/richardtallent/vite-plugin-singlefile
5. TypeScript dokumentáció: https://www.typescriptlang.org/docs
6. Node.js tesztkeret (`node:test`): https://nodejs.org/api/test.html
7. Typst (a dokumentáció szedése): https://typst.app/docs